Interpretable Deep Convolutional Fuzzy Classifier

Mojtaba Yeganejou, Scott Dick, *Member, IEEE*, and James Miller, *Member, IEEE*

***Abstract*—While deep learning has proven to be a powerful new tool for modeling and predicting a wide variety of complex phenomena, those models remain incomprehensible black boxes. This is a critical impediment to the widespread deployment of deep learning technology, as decades of research have found that users simply will not trust (i.e., make decisions based on) a model whose solutions cannot be *explained*. Fuzzy systems, on the other hand, are by design much more easily understood. In this article, we propose to create more comprehensible deep networks by hybridizing them with fuzzy logic. Our proposed architecture first employs a convolutional neural network as an automated feature extractor and then performs a fuzzy clustering in the derived feature space. After hardening the clusters, we employ Rocchio's algorithm to classify the data points. Experiments on three datasets show that the automated feature extraction substantially improves the accuracy of the fuzzy classifier, and while the substitution of a fuzzy classifier slightly decreases the network's performance, we are able to introduce an effective interpretation mechanism.**

***Index Terms*—Deep learning, explainable artificial intelligence (XAI), fuzzy logic, neuro-fuzzy systems.**

I. Introduction

DEEP neural networks have become the most effective approach to problems, including image recognition tasks [1]–[4], natural language processing [5]–[8], speech recognition [9]–[11], and many others. AlphaGo's defeat of champion human players [12] has seemingly become emblematic of deep learning's promise in the public eye. The demand for deep learning in the corporate world is so strong that NVIDIA saw its stock prices double in 2017 [13], valuing the company at $178.5 billion in October 2018.

However, a large-scale deep neural network is a distributed pattern of millions of connections in several layers, containing tens of thousands of neurons [14]. This makes the model completely uninterpretable for humans, while research shows that users are not willing to trust such a model for critical decisions [15]–[17]. Explanation mechanisms are a critical design element of practical intelligent systems, and research into explanations for deep networks (often called eXplainable Artificial Intelligence, XAI) is an area of intense interest.

Manuscript received December 1, 2018; revised May 5, 2019 and September 5, 2019; accepted September 27, 2019. Date of publication October 9, 2019; date of current version July 1, 2020. This work was supported in part by the Natural Science and Engineering Research Council of Canada under Grant RGPIN 2017-05335. *(Corresponding author: Scott Dick.)*

The authors are with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB T6G 2R3, Canada (e-mail: yeganejo@ualberta.ca; dick@ece.ualberta.ca; jm@ece.ualberta.ca).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TFUZZ.2019.2946520

Interpretability is not a new problem in the field of neural networks. Explanation mechanisms were demanded for earlier generations of shallow neural networks as well. One strand of this research focused on extracting interpretable rules from trained networks; see [18] for a discussion. There is a small amount of research continuing this line of inquiry in deep networks. The authors of [19] and [20] have introduced rule extraction mechanisms, while the authors of [21]–[23] used visualization techniques to interpret the network's decisions. Zintgraf *et al.* [24] also suggest that the saliency of a given pixel in an input image can be determined by estimating the change in classifications resulting from its absence.

Another approach was to make neural networks interpretable by design; fuzzy neural networks (e.g., [25]) and neuro-fuzzy systems (e.g., [26]) were prominent among these efforts [27]. Our approach to XAI is to continue this second approach, creating hybrids of deep learning and fuzzy logic (we will henceforth refer to such architectures as "deep fuzzy systems.") Our overarching research problem is thus to create deep fuzzy systems, which remain as accurate as existing models, while being substantially more interpretable. In this article, our objective is to design and evaluate such a system.

We presented a preliminary evaluation of a deep fuzzy classifier, which combines fuzzy c-means (FCM) clustering and a convolutional neural network (CNN) for image classification in [28]. The CNN is used as an automated feature extraction algorithm [29]; the fuzzy clusters (after hardening) are employed as a classifier using Rocchio's algorithm [30]. (Note that we could easily modify this classifier for regression and function approximation by replacing Rocchio's algorithm with, e.g., the fuzzy nearest-prototypes algorithm [31].) Our contributions in this article are a thorough evaluation of that architecture across additional datasets, comparing it with an extension employing the Gustafson–Kessel (GK) [32] fuzzy clustering algorithm, and demonstrating the interpretability of the resulting classifier on one of the datasets.

To evaluate the system, we have chosen the MNIST [33], Fashion MNIST [34], and CIFAR-10 [35] datasets. All three are computer vision datasets, including ten classes of grayscale digits, grayscale clothes, and RGB objects, respectively. We find that the deep fuzzy classifier is somewhat less accurate than the original deep networks for each dataset, but more easily interpreted.

The remainder of this article is organized as follows. We review essential background and related work in Section II. We present our proposed architecture in Section III. We describe our experimental methodology in Section IV and present our results in Section V. In Section VI, we demonstrate the interpretability

of the deep fuzzy systems on the MNIST dataset, and we offer a summary and discussion of future work in Section VII.

II. Background Review

A. Deep Learning, Clustering, and Fuzzy Systems

There is a small amount of prior work in hybridizing deep neural networks and clustering algorithms. Xie *et al*. [36] combined stacked denoising autoencoders with a k-means variant using Kullback–Leibler divergence as an objective function. They presented a way to learn a representation specialized for clustering without ground truth cluster membership labels. Tian *et al*. [37] presented a novel graph clustering method via deep learning. They used sparse autoencoders for embedding of the graph data in a real-valued manifold and then clustered the data with the k-means algorithm. Maaten [38] presented a general dimensionality-reduction technique by employing restricted Boltzmann machines for learning low-dimensional embeddings. De la Rosa and Wen [39] also used a restricted Boltzmann machine as a feature extractor for a probabilistic clustering algorithm and fitting fuzzy rules over the clusters.

In the same way, there is little current research in hybridizing fuzzy logic and deep learning, even though fuzzy neural networks and neuro-fuzzy systems have been significant research topics for over 25 years [27]. Aviles *et al*. [40] combined an ANFIS ensemble and a recurrent network using long short-term memory components to estimate contact forces on tissues during remote surgical procedure. Chen *et al*. designed a fuzzy restricted Boltzmann machine for improved representation capability and robustness in [41]. Zheng *et al*. extend this architecture to Pythagorean fuzzy sets in [42], while Shukla *et al*. instead extended it to interval type-2 fuzzy sets in [43]. Zheng *et al*. hybridized Pythagorean fuzzy sets and stacked denoising autoencoders in order to enhance representation abilities, robustness, and accuracy in industrial accident warnings in [44]. Zhou *et al*. and Rajurkar and Verma [45] design stacked Takagi–Sugeno–Kang (TSK) fuzzy systems. Tan *et al*. use fuzzy logic to reduce the memory requirements of a CNN by proposing a one-shot fuzzy qualitative deep compression method for pruning the redundant parameters [46]. Recently, Zhou *et al.* [47] have introduced the highly interpretable deep TSK fuzzy classifier based on shared linguistic fuzzy rules and a stacked hierarchical structure of TSK fuzzy classifiers. John *et al.* [48] used FCM clustering for image segmentation as a preprocessor before a deep convolutional network. Other than [39], which uses restricted Boltzmann machines and fuzzy modeling for performing probability-based clustering, however, the combination of deep learning and fuzzy clustering is currently unexplored.

B. Fuzzy Clustering

Clustering algorithms are generally defined as algorithms that are used for identifying groups of objects, which are more similar to each other in comparison with the objects outside of the group. Clustering takes the form of identifying feature-space collections of those objects exhibiting compactness within the collection and separation from other collections. Literally, thousands of clustering algorithms have been developed in the pattern recognition literature [49]. Fuzzy clustering methods attempt to accommodate outliers, noise, and boundary cases, which may arguably belong to multiple groups. As with fuzzy sets, the membership of a data point in a cluster can be a nonzero value for an arbitrary number of clusters [50].

The FCM algorithm is one of the most famous approaches in fuzzy clustering. Bezdek developed the FCM algorithm as a fuzzification of the well-known k-means algorithm in his Ph.D. thesis [32]. The algorithm optimizes the objective function

$$\text{cost} = \Sigma_{j=1}^{n}\Sigma_{i=1}^{c} u_{ij}^{m} \left\|x_j - v_i\right\|_{A_i}^{2} \tag{1}$$

where v_i is the *i*th cluster center, x_j is the *j*th data point, μ_{ij} is the membership of the *j*th data point in the *i*th cluster, *m* is the fuzzifier exponent, and the norm is the Euclidean distance. The FCM algorithm optimizes the objective function using alternating optimization of the centers and membership functions [51].

The Euclidean distance induces hyperspherical clusters. By substituting the Mahalanobis distance, FCM can learn hyperellipsoidal clusters as in the GK [52] algorithm. In this algorithm, each cluster is characterized by a covariance matrix *A* and cluster centers. This matrix induces for each cluster a norm of its own $\|x\|_A := \sqrt{x^T A x}$. In order to avoid a minimization of the objective function by matrices with almost zero entries, a constant volume of clusters is required, i.e., $\det(A) = \rho$, where *p* is the number of features in the dataset. This forms a constrained optimization, which can be solved by the method of LaGrange multipliers. Thus, the objective function can be represented as follows:

$$\text{cost} = \Sigma_{j=1}^{n}\Sigma_{i=1}^{c} u_{ij}^{m} \left\|x_j - v_i\right\|_{A_i}^{2} - \Sigma_{i=1}^{c}\lambda_i \left(\det\left(A_i\right) - 1\right). \tag{2}$$

The covariance matrices are updated during alternating optimization using the following equations:

$$A_i = \sqrt[p]{\det\left(S_i\right)} S_i^{-1} \tag{3}$$

$$S_i = \Sigma_{j=1}^{n} u_{i,j}^{m} \left(x_j - v_i\right)\left(x_j - v_i\right)^T. \tag{4}$$

Both GK and FCM algorithms continue iterations of learning until reaching a stopping condition such as $\|u(k+1) - u(k)\| < \delta$, where δ is a minimum improvement, or a maximum number of iterations [53].

C. Convolutional Neural Networks

LeCun *et al*. [54] developed CNNs for image recognition. The key insight behind the CNN is to take advantage of the *N*-dimensional ($N = 2$, for two-dimensional images) local structure surrounding every pixel. Quite obviously, the value (i.e., RGB, or whatever other color space is used) of a given pixel is only meaningful to the human eye in relation to the values of the surrounding pixels. Some research indicates that human vision is tuned to recognize scenes having a relatively narrow band of spatial statistics; for a discussion, see, e.g., [55]. The CNN takes advantage of these characteristics by defining "convolutional" layers, in which groups of neurons implement convolutions over their input (either an image at the first layer, or the outputs

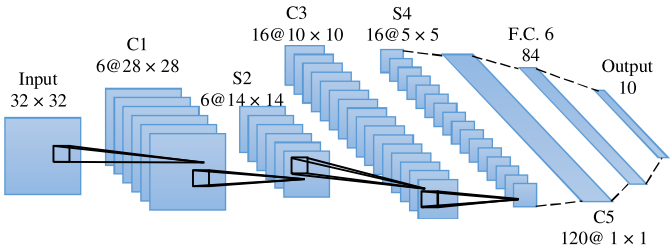

Fig. 1. LeNet architecture.

of another layer as signals progress deeper into the network) [54]. These convolutions, as synopses of the local neighborhood around a pixel, seem to produce "better" features (in the sense of superior discrimination between classes). Stacking multiple convolutional layers in sequence seems to further improve the resulting features.

A convolution operation can be carried out by defining a collection of neurons, whose spatial extents overlap and cover the entirety of the image, while replicating the operation in question. These groups are referred to as planes, and their outputs feature maps. Replication of the same operation is achieved simply by constraining the weight vectors for every neuron in a plane to be duplicates of each other. The resulting feature map is then the outputs of the given convolution, applied to the input image (e.g., edge detection). We expect that there will be multiple planes for an image, yielding multiple feature maps that will be passed to the next layer. Convolutional layers are commonly followed by subsampling layers. This will reduce the resolution of the feature map and thus sensitivity of the output to shifts and distortions, while allowing for the detection of image elements with larger spatial extents [54]. After all of these transformations, the final set of features then passes to a final classification layer (e.g., a fully connected layer of sigmoid neurons). Empirically, feature extraction via convolutions is currently the leading approach for building accurate neural models [1], [22], [54], [56].

Fig. 1 shows the architecture of the "LeNet" from [54]. Layer C1 in Fig. 1 is the first convolutional layer, whose neurons receive a group of individual pixel values from a small spatially coherent subset of the image. A plane is thus a collection of neurons whose input-domain spatial extents (i.e., the convolution windows) overlap and cover the entirety of the image.

The general form of the convolution operation is

$$v_{ij}^{l} = \Sigma_{a=0}^{m-1} \Sigma_{b=0}^{m-1} w_{ab} X_{(i+a)(j+b)}^{l-1} \tag{5}$$

where the convolution window is of size $m \times m$, X_{ij} is the ijth pixel in the current subimage, and w_{ab} is the weight of the connection between the pixel at offset (a, b) and the current neuron. Within a plane, all weights w_{ab} for offset (a, b) are shared between all neurons. Once the convolution is completed, the result is then passed through a nonlinear activation function; the most common choice in modern CNNs is the rectified linear unit (ReLU) function

$$y_{ij}^{l} = \max(v_{ij}^{l}, 0). \tag{6}$$

The next layer, S2, is a subsampling or "pooling" layer, which maps small neighborhoods in each feature map output to single pixels. Two common approaches are max-pooling and mean-pooling

$$y_{ij}^{l} = \max_{a} \max_{b} \left(X_{(i+a)(j+b)}^{l-1} \right), \quad a, b = (1, \ldots, k) \tag{7}$$

$$y_{ij}^{l} = \left(\sum_{c=1}^{k} \sum_{d=1}^{k} X_{(i+c)(j+d)}^{l-1} \right) \cdot (\omega_{ij}/k^2) + b_{ij} \tag{8}$$

where ω_{ij} is a weight coefficient, b_{ij} is a bias term, ω_{ij} and b_{ij} are adaptable, and k is the convolution window size.

Additional layers can be added to CNNs to realize additional properties. For instance, local response normalization (LRN) layers have been added to CNNs in order to implement the concept of lateral inhibition from human vision [1]. Lateral inhibition is the capacity of an excited neuron to reduce the activity of its neighbors. One form of lateral inhibition is given by [1]

$$b_{x,y}^{i} = \frac{a_{x,y}^{i}}{\left(k + \alpha \Sigma_{j=\max\left(0, i-\frac{n}{2}\right)}^{\min\left(N-1, i+\frac{n}{2}\right)} \left(a_{x,y}^{i} \right)^2 \right)^{\beta}} \tag{9}$$

where $a_{x,y}^{i}$ is the activity of a neuron computed by applying kernel i at position (x, y) and then applying the ReLU nonlinearity, N is the total number of kernels in the layer, n is number of adjacent kernel maps at the same spatial position, k is a bias, and α and β are scaling factors.

D. Dimension Reduction

Principal component analysis (PCA) [57] is a standard linear dimensionality reduction technique. Also known as the discrete Karhunen–Loève transform [58], PCA projects the input data x into a lower dimensional space spanned by selected eigenvectors of the covariance matrix for the given dataset. Usually, the eigenvectors associated with the p largest eigenvalues of the matrix are selected.

A recent alternative is the uniform manifold approximation and projection (UMAP) algorithm [59]. The theoretical foundation is to use local manifold approximations, patched together based on the associated fuzzy simplicial sets (a concept from topological theory). One constructs a graph representation of the dataset and finds a low-dimensional embedding that preserves the graph structure. For each data point $\vec{x}_i$ in the N-element dataset D, first find its n nearest neighbors $\vec{x}_{ij}, j = (1, \ldots, n)$ and then compute [59]

$$\rho_i = \underbrace{\min}_{j} \left\{ d\left(\vec{x}_i, \vec{x}_{ij}\right) \right\} \tag{10}$$

and determine σ_i such that

$$\sum_{j=1}^{n} \exp\left(\frac{-\max\left(0, \left\{ d\left(\vec{x}_i, \vec{x}_{ij}\right) \right\} - \rho_i \right)}{\sigma_i} \right) = \log_2(n). \tag{11}$$

These two values will yield a (smoothed) normalized distance between that point and its nearest neighbors; note that each nearest neighbor may well have a different distance to this

given point in their own local manifold. We can now construct the weighted graph G, whose vertices are the points $\vec{x}_i$, with edges $(\vec{x}_i, \vec{x}_{ij})$, and edge weights $\exp(\frac{-\max(0,\{d(\vec{x}_i,\vec{x}_{ij})\}-\rho_i)}{\sigma_i})$. As we iterate from $i = 1, \ldots, N$, edge weights are combined using a fuzzy union; the probabilistic t-conorm is recommended based on empirical study. Forming the graph G in this manner is equivalent to computing and patching together the fuzzy simplicial sets for each local manifold. This is a topologically sound method of representing the dataset even though the local manifolds almost certainly have incompatible metrics [59].

Once the graph is constructed, we find a topology-preserving low-dimensional embedding of G. We first compute the graph Laplacian L. Form the weighted adjacency matrix A and degree matrix DM of G and compute

$$L = \mathrm{DM}^{0.5} \cdot (\mathrm{DM} - A) \cdot \mathrm{DM}^{0.5}. \tag{12}$$

Then, we find a low-dimensional embedding (again, fuzzy simplicial sets) from the Laplacian with a minimum cross-entropy loss with respect to the fuzzy memberships compared to G. This is achieved by stochastic gradient descent; the (differentiable) embedding form is adapted from force-directed graph layout algorithms, as [59]

$$\xi(x, y) = \left(1 + a \cdot \left(\|x - y\|_2^2\right)^b\right)^{-1}. \tag{13}$$

For more details, see [59].

E. Explainable Artificial Intelligence

Modern deep learning systems can match or even exceed human performance in some tasks. However, they are black boxes, and thus, their decision-making process is sometimes incomprehensible to even human experts in the area. For example, consider the 37th move in the second game of the historic Go match between Lee Sedol, a top Go player, and AlphaGo [60] built by DeepMind. Another Go expert, F. Hui, commented, *"It's not a human move. I've never seen a human play this move."*

The simple fact is that all of the advances made in AI in the last two decades could be wasted if humans cannot *trust* those algorithms to make decisions at some level. Research has repeatedly shown that providing users with an explanation for decisions made by an algorithm is an essential precondition to winning user trust [15], [16]. In addition, explanation mechanisms enable other key system characteristics [61].

1) *Verifiability:* The AI model just infers from the data; any biases in the data are thus faithfully reproduced by the model [62]. It is also possible for the model to be incorrect; software is, after all, well known for quality problems. Thus, verification of the system by domain experts is required. This, however, is only possible if those experts can *understand* the model.
2) *Comparative judgments:* Interpreting a model will then allow us to compare and contrast different models. Distinct models may have the same classification performance, but completely different decision-making methods. It is possible that some of those models, while presently reasonable, might foreseeably lead to undesired outcomes in the future.
3) *Knowledge discovery:* As with other data mining algorithms, deep learning can observe data patterns not visible to humans. However, in the knowledge discovery in databases (KDD) framework [63], such patterns must then be presented to an analyst and integrated into operations. Plainly, this means humans must understand them.
4) *Legal compliance:* The European Union has issued regulations, which implement a "right to explanation." Users can demand an explanation of an algorithmic decision impacting them [64]; plainly, an uninterpretable model is unlikely to withstand judicial scrutiny.

F. Network Reversal Techniques

Our proposed explanation mechanism involves determining what network inputs are associated with certain chosen classification results. Multiple methods for "reversing" signals in a neural network have been proposed. Herein, we review layerwise relevance propagation (LRP), Taylor decomposition, and guided backpropagation.

LRP uses local redistribution rules at each neuron to determine a relevance score for each of its inputs given its weight vector and output. These propagate backwards until the system input is reached [65]. Taylor decomposition is very similar to LRP except a different function (Taylor decomposition) is used to compute the relevance of each input to the neuron output [65], [66]. Guided backpropagation extends the deconvolution approach by combining it with a "saliency map" of the image [67]. The deconvolutional network ("deconvnet") is a visualization technique learned by neurons in higher layers of a CNN. Given a high-level feature map, the "deconvnet" inverts the data flow of a CNN. Typically, a single neuron is left nonzero in the high-level feature map. Then, the resulting reconstructed image shows the part of the input image that is most strongly activating this neuron (and hence the part that is most discriminative for it). In order to conduct a reconstruction through max-pooling layers, this method first performs the forward pass calculations to compute "switches"—positions of maxima within each pooling region, and then uses deconvolution, unpooling, and rectification to reconstruct the preceding feature maps. In guided backpropagation, the firing strength of each pixel is visualized in order to produce a given result at one output neuron. The authors showed that this was similar to deconvolutional networks [22], [23] except for the handling of the nonlinearity at ReLUs. Springenberg *et al*. [68] combined these two approaches, along with a rule to zero out the importance signal at a neuron if either the input or the importance signal is negative.

III. Proposed Architecture

Fig. 2 shows a sample of our proposed deep fuzzy architecture, which was initially developed in [28]. The idea is to use the CNN as a feature extraction engine and then pass the outputs of the last convolutional layer to a fuzzy classifier. This implies

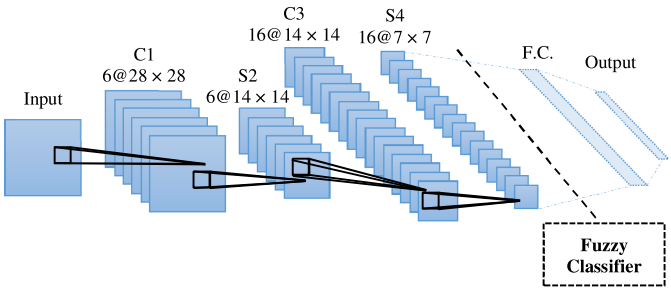

Fig. 2. Sample of our deep fuzzy system.

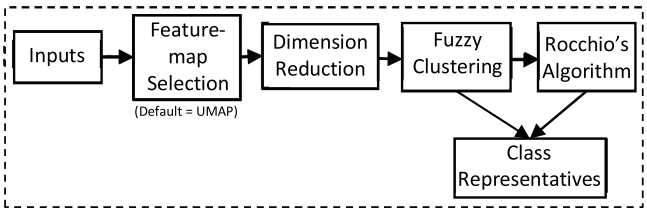

Fig. 3. Fuzzy classifier unit.

that the CNN portion can change based on the dataset and our needs. In Fig. 2, layers C1, S2, and C3 are identical to the LeNet architecture in Fig. 1.

After C3, the data path will be divided into two parts. For the deep fuzzy architecture, the feature maps of C3 will be the derived feature space passed to the fuzzy classifier. A fuzzy clustering is performed, and Rocchio's algorithm is used as the final classifier. The other branch leads through a more traditional deep network (henceforth the *base CNN*); a subsampling layer S4, then a fully connected layer of ReLU neurons, and finally a fully connected (F.C.) layer of softmax neurons, with activation functions

$$y_k = \frac{\exp\left(\overrightarrow{w_k^T}.\overrightarrow{Y_{\mathrm{FC}}}\right)}{\sum_{m=1}^{M}\exp\left(\overrightarrow{w_m^T}.\overrightarrow{Y_{\mathrm{FC}}}\right)} \tag{14}$$

where y_k is the kth network output, $\overrightarrow{Y_{\mathrm{FC}}}$ is the output vector from the previous F.C. layer, $\vec{w}_k^T$ is the transposed weight vector for the kth softmax neuron, M is the number of neurons in the softmax layer, and $\overrightarrow{w_k^T}.\overrightarrow{Y_{\mathrm{FC}}}$ is called the logit value of output layer. The resulting network outputs are confined to [0,1] and sum to 1 for each network input.

Fig. 3 shows a block diagram of the fuzzy classifier unit. Feature map selection means that either all elements of a feature map are selected or none are; this coherence is important as the classifier is intended to be interpretable. This stage omits redundant feature maps to improve classification accuracy.

Dimension reduction based on PCA (as in [28]) or UMAP is then employed as GK clustering requires a matrix inversion step; with each selected feature map yielding 196 features (a 14 × 14 image), clustering in the unreduced dataset could be prohibitively slow. The fuzzy clustering is executed, and Rocchio's algorithm is used as the classifier. Algorithm III-1 presents a fuzzy version of Rocchio's algorithm.

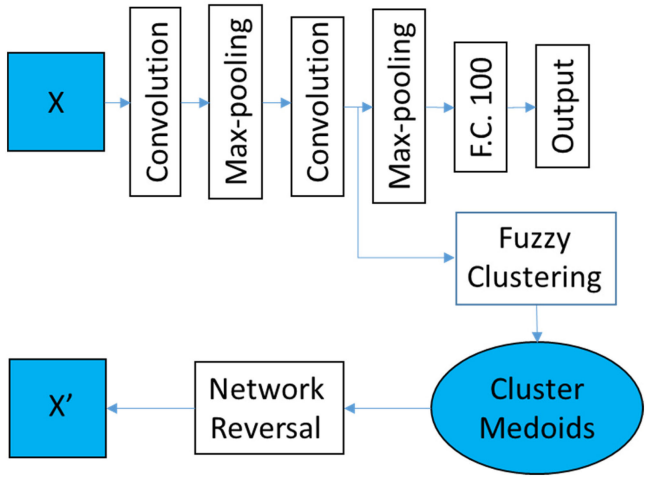

Fig. 4. Data flow for our explanations.

Algorithm III-1: Fuzzy Rocchio's Algorithm.

Start:

1. Calculate memberships via FCM/GK training
2. For each data point assign a class according to the highest membership
3. Choose the most common label in each cluster as the class of cluster
4. Calculate testing_memberships via FCM/GK prediction
5. Perform testing using testing_memberships and training classes

End

The explanation mechanism starts with the clustering; we identify the medoid element of each fuzzy cluster and treat it as the cluster representative [69]. We will then map this element back to the original image space, identifying the pixels that are most relevant in producing the classifier output. Plotting these as a heat map gives us our explanation for that whole cluster of images. Note that, since the medoid is an actual element of the dataset, we do not need to invert the UMAP algorithm to determine the unreduced derived feature vector for it; we need to merely look it up from the example ID. The explanation mechanism is summarized in Fig. 4.

IV. Methodology

A. Research Questions and Experimental Designs

In the following, we pose several research questions and design experiments to answer them. In general, our designs compare the performance of our deep fuzzy system against the same fuzzy classifier in the original image, against the full deep learner, and against recent results in the literature.

Any new learning algorithm must offer some improvement on the status quo; our first research question is thus *RQ1: Does the fuzzy clustering layer improve upon the accuracy of the deep learner it is based on? If not, what is the tradeoff between the accuracy and interpretability?* We will evaluate RQ1 with a

traditional comparison (via out-of-sample accuracy) of the deep fuzzy system against the deep learner it is based on. If (as we suspect) the deep fuzzy system is less accurate than the deep learner, this experiment also reveals the size of that difference.

The next question is whether the deep fuzzy system has in fact given us any advantage over the fuzzy system component itself. This yields *RQ2: Does the use of automated feature extraction lead to improve classification results versus training our fuzzy classifier in the original feature space?* RQ2 can again be evaluated by comparing the deep fuzzy system against the same fuzzy classifier trained in the original image space.

We then turn our attention to the wider literature on deep learning; there are many papers that investigate our benchmark datasets, and we also need to determine how our deep fuzzy system performs against them. This yields *RQ3: Is our proposed architecture comparably accurate to existing deep learning systems?* Again, this question is answered by comparing our deep fuzzy system to those other architectures, including the state of the art (as best we can determine).

Our intent is that the deep fuzzy architecture of Fig. 2 could be applied to any CNN, not just LeNet and its variations. Our next question is *RQ4: Can our deep fuzzy architecture be generalized to other CNNs?* We will examine this question by combining the fuzzy classifier with four different CNNs, and repeating the experiments for RQ1 on each. These four networks include variants on LeNet and AlexNet, a CNN with four convolutional layers, and residual networks (ResNets).

We can also inquire what the impact of different clustering algorithms might be. FCM produces hyperspherical clusters, while GK clusters are hyperellipsoidal. Is there a performance difference between them? (Note that these are only two of the thousands of fuzzy clustering algorithms that have been proposed.) This yields *RQ5: What is the impact of different clustering algorithms on the deep fuzzy system?* RQ5 can be evaluated by substituting different fuzzy clustering algorithms in Fig. 3 and then comparing the resulting deep fuzzy systems using the same design as RQ1.

Finally, the prime intent of designing this deep fuzzy system is to obtain a more interpretable classifier. We thus pose research question *RQ6: Is the deep fuzzy system more interpretable than the original deep fuzzy system?* Formal experiments to measure a characteristic such as interpretability are a major research topic in the psychology literature and are beyond the scope of this article. However, we can offer a demonstration of our proposed explanation mechanism and explore how it accounts for both correct and erroneous classifications in our data.

B. Datasets

We have chosen three commonly used benchmark datasets for deep learning: MNIST [33], Fashion MNIST [34], and CIFAR-10 [1]. MNIST is a collection of grayscale images, each representing one handwritten digit and labeled with the correct value. The samples were gathered from roughly 250 individuals; one half are U.S. Census Bureau employees, while the other half are high school students. MNIST is prepartitioned into 60 000 training samples and 10 000 testing samples; the writers for the two groups are disjoint. Each MNIST image is a 28×28 pixel square, centered on the center of mass of the digit pixels. These images are constructed from a set of 20×20 pixel square images containing the actual digits, size-normalized to this bounding box. While the original samples were binary images, the antialiasing technique used in the normalization introduced gray levels. See [33] for more details.

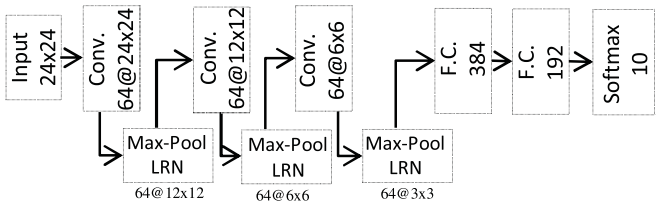

Fig. 5. Base CNN for CIFAR-10.

Fashion MNIST shares the same image size and structure of training and testing splits with MNIST. Each Fashion MNIST sample is a 28×28 grayscale image, associated with a label from one of ten classes (t-shirts, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots) [34].

CIFAR-10 [1] is an RGB image dataset, consisting of 60 000 images of size 32×32 in ten mutually exclusive classes, with 6000 images per class, again prepartitioned into training and testing sets. There are 50 000 training images and 10 000 testing images.

C. Experimental Setup

The existing literature on all three datasets reports results based on a single-split design; for comparability, we will follow this design, employing the same test set of 10 000 images as our out-of-sample evaluation. Our training datasets will be 10 000 examples randomly selected from the predefined training sets, to reduce the computation time. For MNIST and Fashion MNIST, we employ the CNN of Fig. 2. However, preliminary experiments showed this architecture to perform poorly on CIFAR-10, so we instead use a variant of the CNN in [1] this dataset; the architecture is depicted in Fig. 5. Note that we have cropped each sample image to 24×24 image. That is a common technique to improve performance on CIFAR-10 [70].

To train the networks, we employed the mini-batching form of stochastic gradient descent, with an added momentum term

$$\Delta w_{ij}(n) = \eta \cdot \delta_j(n) \cdot x_i(n) + \alpha \cdot \Delta w_{ij}(n-1) \tag{15}$$

where $\Delta w_{ij}(n)$ is the update for the $ij\text{th}$ weight after observing the $n\text{th}$ mini-batch of input patterns, η is the learning rate, $\delta_j(n)$ is the error gradient for neuron j, $\Delta w_{ij}(n-1)$ was the update the $ij\text{th}$ weight after the previous mini-batch, and α is the momentum constant. Our loss function is the cross-entropy

$$\epsilon = -\sum_{c=1}^{C} d_c \log\left(p\left(y_c|\vec{x}\right)\right) \tag{16}$$

where d_c is a binary indicator of whether the ground-truth class for the current input $\vec{x}$ is c, and p is the probability that the output y_c will be predicted by the deep learner for input $\vec{x}$ [14]. In our experiments, we first train the full deep convolutional network (the CNN and F.C. parts of Fig. 2) and test it on our

TABLE I
COMPARISON OF OUR ACCURACY AND MOST RECENT ACCURACIES ON MNIST

Method	Number of Clusters	Accuracy
Our experiments		
DEEP_FCM	100	96.92
DEEP_FCM	10	87.63
DEEP_GK	10	96.97
DEEP_GK	100	97.36
Deep CNN	-	98.78
FCM	100	84.48
FCM	10	60.92
GK	10	70.92
GK	100	81.61
Published results		
[72]	-	95.0
[36]	-	84.30
[71]	-	99.79
[73]	-	95.0

TABLE II
COMPARISON OF OUR ACCURACY AND MOST RECENT ACCURACIES ON FASHION MNIST

Method	Number of Clusters	Accuracy
Our experiments		
DEEP_FCM	100	79.69
DEEP_FCM	10	64.93
DEEP_GK	10	74.33
DEEP_GK	100	81.71
Deep CNN	-	86.51
FCM	100	74.24
FCM	10	58.06
GK	10	53.8
GK	100	73.1
Published results		
[75]	-	58.64
[76]	-	90.03
[1]	-	86.43
[74]	-	96.8

holdout data. Then, we copy the outputs of Layer C3 for each element of the training and test sets; this gives us the feature space we will use for our fuzzy classifier. For dimensionality reduction, we have used PCA with 17 dimensions for MNIST, and UMAP with three dimensions for Fashion MNIST and CIFAR-10. We execute the fuzzy clustering algorithm multiple times, varying the number of clusters from 10 to 100 inclusive (a typical design for clustering experiments [53]). Our cluster validity criterion is the training error determined from applying Rocchio's algorithm on the training set.

We have decided not to optimize the hyperparameters of the CNN component of our design, in order to focus on the impact of the fuzzy component on our results. The CNNs are thus trained with a mini-batch size of 128, $\eta = 0.01$, and $\alpha = 0.9$. While it may be the case that further optimization of the CNN might increase the overall performance of our deep fuzzy classifier, in this article, we are more interested in the contribution of the fuzzy component and the interpretability. In all tables, we compare our out-of-sample accuracy (from the clustering with the lowest training error) with the reported accuracy from the literature.

V. EXPERIMENTAL RESULTS

A. Performance on MNIST

Our out-of-sample test accuracies on MNIST are reported in Table I (this is always the traditional classification accuracy). This table is divided into two parts. The first part reports the performance of our deep fuzzy classifiers ($m = 1.1$), deep CNN, and fuzzy classifiers ($m = 1.1$) in the original image on MNIST. The second part of table shows the results of other recent methods including the state-of-the-art method [71].

B. Performance on Fashion MNIST

Our out-of-sample performance on Fashion MNIST is reported in Table II, which is organized in the same way as Table I.

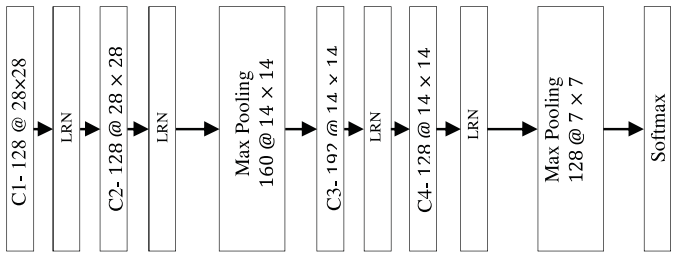

Fig. 6. Deeper CNN for Fashion MNIST classification.

TABLE III
COMPARISON OF OUR ACCURACY AND MOST RECENT ACCURACIES ON FASHION MNIST

Method	Accuracy
CNN of Figure 6	92.37%
Deep GK with 10 clusters	92.03%

Similar to MNIST, the fuzzifier m has been set to 1.1 in order to reach greatest accuracy. The current state of the art on this dataset appears to be [74].

Plainly, the CNN in our experiments falls far short of the current state of the art. We have thus evaluated a CNN with four convolutional layers (each followed by an LRN layer) on Fashion MNIST. The architecture is shown in Fig. 6, and we train the network with the full 60 000-exmple training set. The learning rate is also adaptive instead of constant. As before, we extract the induced features from the CNN immediately before the final max-pooling layer and pass them to our fuzzy classifier. We report the results of a GK fuzzy clustering with ten clusters in Table III. Both architectures have substantially improved from our earlier results, although they are still not quite a match for the state of the art. Importantly, the drop-off from the CNN to our corresponding deep fuzzy system is again only minor. Note that here the fuzzifier has been set to $m = 1.6$.

TABLE IV
COMPARISON OF OUR ACCURACY AND MOST RECENT ACCURACIES ON CIFAR-10

Method	Number of Clusters	Accuracy
Our experiments		
DEEP_FCM	100	59.29
DEEP_FCM	10	49.04
DEEP_GK	10	74.07
DEEP_GK	100	72.05
Deep CNN	-	80.7
FCM	100	22.66
FCM	10	31.66
GK	10	28.59
Published results		
[81]	-	79.7
[82]	-	96.53
[83]	-	82.18
[1]	-	89
[77]	-	98.53

TABLE V
COMPARISON OF OUR ACCURACY AND MOST RECENT ACCURACIES ON CIFAR-10

Method	Accuracy
ResNet56 [80]	92.67%
Deep GK with 10 clusters	91.87%

C. Performance on CIFAR-10

We report our out-of-sample performance on CIFAR-10 in Table IV, which is again organized in the same fashion as Tables I and II. The current state of the art appears to be [77].

Again, our base CNN was not effective on CIFAR-10. We have, therefore, repeated our experiments with the more modern ResNets [78] architecture. He *et al*. introduced the second version of ResNets in [79]; we use this second version, training a ResNet56 on CIFAR-10 using the implementation in [80].

We train the network with the full 50 000-example training set, and the learning rate is adaptive. We compare the results of the ResNet56 against our fuzzy clustering approach as applied to this new network (again, GK fuzzy clustering with ten clusters, and $m = 1.9$) in Table V. We again note an improvement in performance for both architectures, with the deep fuzzy system again only experiencing a minor drop-off.

D. Discussion

We now consider research questions RQ1–RQ5 in light of our experimental results. We defer a discussion of RQ6 until the next section.

RQ1: Across all of our datasets and CNN architectures, we consistently find that the deep fuzzy system is less accurate than the base CNN. The size of this difference varies considerably; in Tables I, III, and V, the difference is less than 1.5 percentage points, but rises to almost 5 and over 6 percentage points in Tables II and IV, respectively. We note that for the latter two, the base CNN itself performs fairly poorly; one possibility is that the CNN features extracted are of a lower discriminative quality. The fact that changing the CNN yields better performance, and a lower difference between the base CNN and the deep fuzzy systems in Tables III and V tends to support this contention.

RQ2: The deep fuzzy systems are unequivocally superior to the fuzzy systems applied to the original images. This is true for both clustering algorithms at every number of clusters tested, across all datasets.

RQ3: From RQ1, we have found that the deep fuzzy systems do not perform as well as the base CNNs. As these base CNNs themselves have not quite matched the state of the art, this question must necessarily be answered in the negative; the deep fuzzy systems are not currently comparable in accuracy to the state of the art.

RQ4: As discussed, the experiments reported in Tables I–V used four different base CNNs. While the efficacy of deep fuzzy systems varied across datasets and architectures, it seems plain that the strategy of passing outputs from the CNN as features to the fuzzy classifier is generally applicable.

RQ5: The fuzzy clustering algorithm chosen does indeed seem to have a substantial impact. For all three datasets, our best results for the deep fuzzy system in Tables I, II, and IV are obtained with the GK algorithm rather than FCM. Note that both of these algorithms belong to the general k-means family; in the future, we suggest that algorithms from other families (e.g., hierarchical, agglomerative, mixture models) should be examined.

VI. INTERPRETABILITY

We now explore RQ6 by demonstrating our proposed explanation mechanism on the MNIST dataset. In the previous sections, we described how k-means-based algorithms yield medoid elements that we can consider representatives of each class. Each cluster medoid is an input image whose average dissimilarity to all other images in the cluster is minimal. Fig. 7 shows the medoids of each class for the MNIST dataset in a ten-way clustering (we selected this clustering for simplicity in this demonstration).

A. Interpreting the Medoids

As discussed, our explanation mechanism determines the saliency of each pixel in the input space for the output class. We will determine those saliencies through guided backpropagation, Taylor decomposition, and LRP, yielding three *saliency maps* for each medoid. We will then investigate the network's errors by both examining the logit values from the softmax layer in the base CNN and comparing the bitmaps of selected misclassified images to the relevant saliency maps.

In the guided backpropagation method, pixels in the saliency maps vary from white to black; these extremes denote the same impact on classification, but different meanings. White pixels are most strongly associated with white in the original image (which are always relevant), and black ones indicate pixels that are both black and highly relevant in the original image. Gray (neutral) pixels have little relevance (note that the bulk of

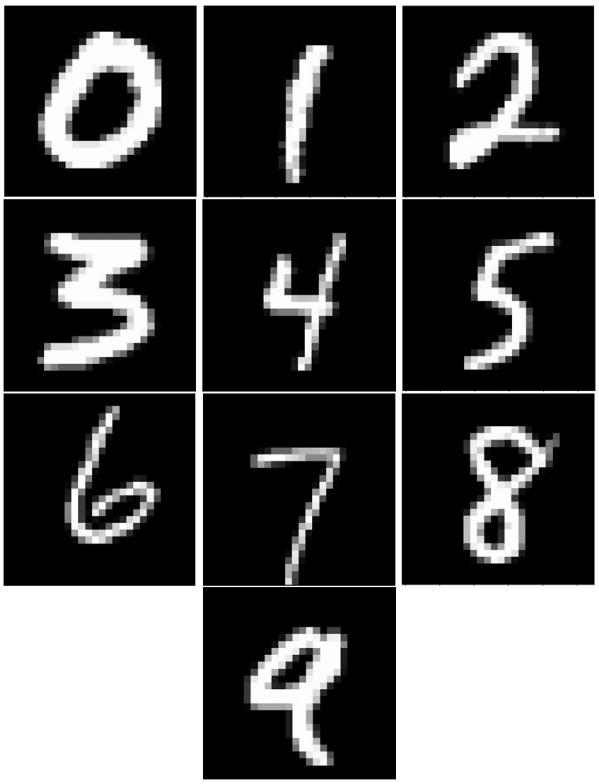

Fig. 7. Medoid elements of each MNIST digit class.

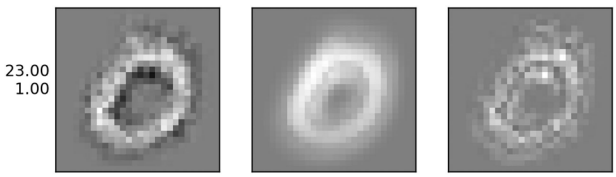

Fig. 8. Results of interpretation techniques for medoid representative of class 0.

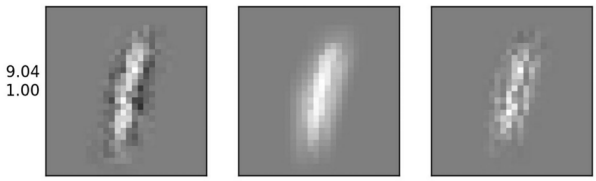

Fig. 9. Results of interpretation techniques for medoid representative of class 1

these are black). Taylor decomposition and LRP, however, only indicate the degree of relevance. Figs. 8–17 present the saliency maps for each class. Guided backpropagation is placed at the left, Taylor decomposition in the middle, and LRP at the right. In each figure, there are two values to the left of the guided backpropagation map. The top value is the logit value for the medoid in the original deep network (value of the output layer before entering the softmax gate), and the bottom value is the probability computed for the medoid image at the correct output neuron.

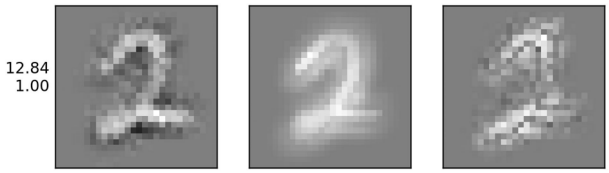

Fig. 10. Results of interpretation techniques for medoid representative of class 2.

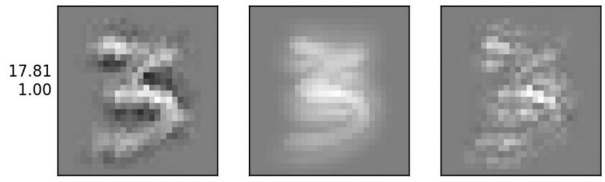

Fig. 11. Results of interpretation techniques for medoid representative of class 3.

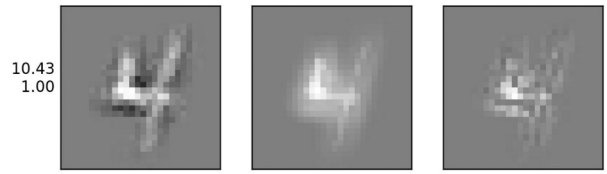

Fig. 12. Results of interpretation techniques for medoid representative of class 4.

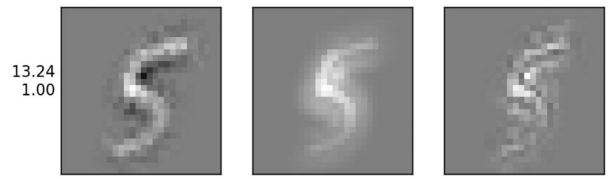

Fig. 13. Results of interpretation techniques for medoid representative of class 5.

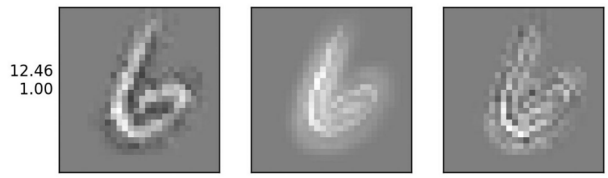

Fig. 14. Results of interpretation techniques for medoid representative of class 6.

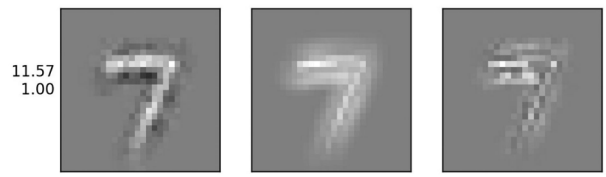

Fig. 15. Results of interpretation techniques for medoid representative of class 7.

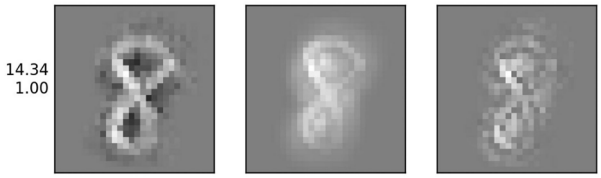

Fig. 16. Results of interpretation techniques for medoid representative of class 8.

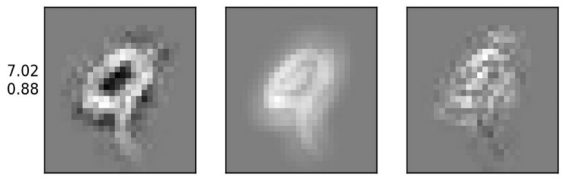

Fig. 17. Results of interpretation techniques for medoid representative of class 9.

TABLE VI
CONFUSION MATRIX OF LOGIT VALUES FOR MEDOID REPRESENTATIVES

	0	1	2	3	4	5	6	7	8	9
medoid 0	23	-3.36	3.67	0.83	-6.17	3.95	5.68	-1.2	0.95	1.25
medoid 1	-0.77	9.04	-1.73	-2.3	-2.33	-2.43	0.63	1.47	2.76	-1.26
medoid 2	1.29	2.77	12.84	4.54	-1.95	-6.81	-5.36	2.89	3.31	-1.33
medoid 3	0.4	-2.9	2.98	17.81	-5.58	10.97	-4.73	-0.19	0.18	-0.43
medoid 4	-5.79	-0.28	-2.67	1.85	10.43	1.63	-1.4	2.57	1.09	2.65
medoid 5	-1.52	-2.8	-2.82	5.74	-2.42	13.24	0.47	-3.02	3.57	2.61
medoid 6	2.69	-0.6	1.22	-0.78	0.3	-0.04	12.46	-5.6	-1.06	-5.9
medoid 7	1.44	2.11	3.87	2.79	-3.39	-1.43	-7.55	11.57	-0.56	3.76
medoid 8	0.61	0.24	3.28	2.93	-4.91	2.23	-2.2	-4.52	14.34	-0.08
medoid 9	0.7	-0.32	0.77	1.26	2.97	-3.64	-3.73	4.25	3.95	7.02

Fig. 18. Misclassified digit (prediction = 2, truth = 7).

B. Misclassifications

To begin our investigation of misclassification errors, we examine the logit values (inputs to the softmax layer) shown in Table VI. Columns in the table are associated with specific output neurons (one per class, following the 1-of-*n* coding), while rows are associated with each of the medoid images. Cells highlighted in green are the highest logit value for each medoid, while lime cells are the second highest.

Consider for example the medoid of "7." The next highest logit values are for classes "2" and then "9" (the difference between them being very small). Fig. 18 shows a "7" input misclassified as "2." The main difference between this image and medoid 7 is the horizontal line in the middle of image. Per Fig. 15, the ending part of the straight line in medoid 7 is less relevant than other elements, while the tail in medoid 2 (see Fig. 10) appears very relevant.

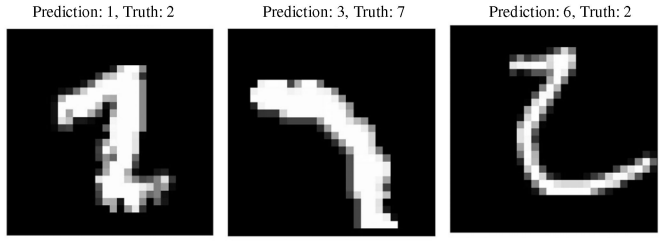

Fig. 19. Hard to recognize digits for humans.

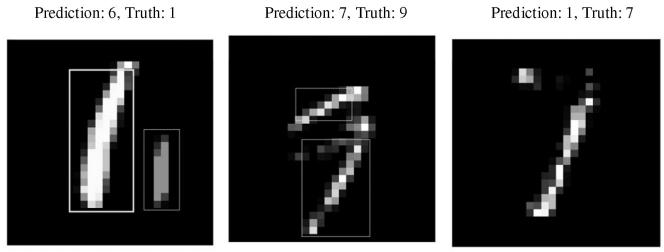

Fig. 20. Corrupted images.

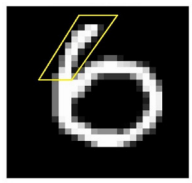

Fig. 21. Misclassified digit sample. Predicted as 0, Truth = 6.

Let us next consider some of the input images that might be problematic even for humans. Some examples are presented in Fig. 19. The leftmost is a "1" misclassified as "2," the middle is a "7" misclassified as "3," and the last is a "2" misclassified as "6." In Fig. 20, we see examples of apparently corrupted images. For these, we highlight very relevant pixels from the saliency maps that indicate why these images were misclassified. The leftmost is a "1" misclassified as "6," the middle is a "9" misclassified as "7," and the last is a "7" misclassified as "1." In all six of these images, we can see that the predictions seem quite reasonable when we examine Figs. 7–16.

Finally, let us consider some examples for which humans would likely have little trouble correctly identifying the digit, e.g., Fig. 21.

For each misclassified digit, we will superimpose it (in red) on the medoid images of the predicted and ground truth classes. (We will just use the Taylor decomposition map for this visualization.) In Fig. 22, we see that Fig. 21 matches the saliency map for "0" better than the one for "6."

Another misclassified digit is shown in Fig. 23. This image is properly a "9" but is misclassified as a "1." Fig. 24 shows that this image is a closer match to the saliency map for "1" than for "9."

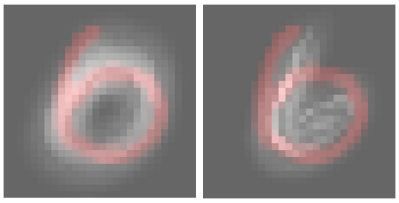

Fig. 22. Superimposing the sample on medoids of class 0 and 6.

Fig. 23. Misclassified digit sample. Predicted as 1, Truth = 9.

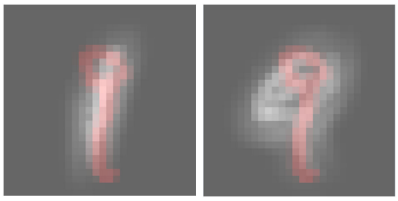

Fig. 24. Superimposing the sample on medoids of classes 1 and 9.

Fig. 25. Misclassified digit sample. Predicted as 7, Truth = 2.

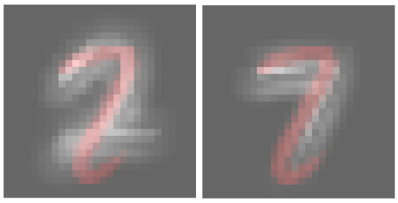

Fig. 26. Superimposing the sample on medoids of classes 2 and 7.

Two more examples are shown below. A "2" is misclassified as "7" in Figs. 25 and 26, while a "5" is misclassified as a "6" in Figs. 27 and 28.

C. Discussion

We now consider research question RQ6 in light of our analysis above. Our results in this section do suggest that the deep fuzzy system is more interpretable than the base CNN used for the MNIST dataset, and furthermore that guided backpropagation appears to yield the most interpretable heat maps.

Fig. 27. Misclassified digit sample. Predicted as 6, Truth = 5.

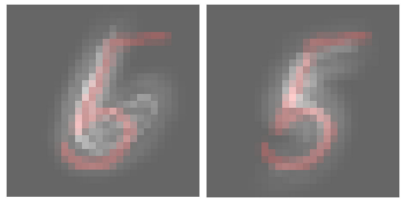

Fig. 28. Superimposing the sample on medoids of classes 6 and 5.

However, the results in this section are properly a case study and, thus, cannot logically be generalized beyond their native context. Furthermore, we have not attempted to rigorously define "interpretability" beyond its intuitive use in recognizing an image. The authors of this article (and likely most of the readers as well) are not representative of the general public, and therefore, what seems clearly interpretable to ourselves might not be so viewed by others. Thus, we urge caution in extending these results to any other context.

VII. Conclusion

In this article, we have proposed and evaluated a novel hybrid of deep learning and fuzzy logic. This "deep fuzzy system" employs CNNs as automated feature extractors, but substitutes a fuzzy classifier for the neural classifier normally used in the CNN architectures. The dataset is clustered in the derived feature space defined by the outputs of the last convolutional layer in the CNN, and then, Rocchio's algorithm is used to infer the class of each new data point. An explanation mechanism for the network is then easily constructed by determining the medoid of each cluster and determining the relevance of each pixel in the input image to classifying that medoid. In experiments on three benchmark datasets, we find that the accuracy of the deep fuzzy system is somewhat lower than that of the deep learner itself. However, a case study of the MNIST dataset shows that our proposed explanation mechanism does seem to account for the network's decisions in an intuitive manner.

In future work, we plan to address two key questions. First, a systematic evaluation of our proposed explanation mechanism needs to be performed; a key question is whether the explanations generated are comprehensible only to domain experts, or if they are accessible to the general public (and could thus fulfill the EU regulations on the right to an explanation). Second, we will study the use of more powerful fuzzy classifiers in the deep fuzzy system. For instance, using the fuzzy clusters to initialize an ANFIS network, which is then trained to minimize classification errors, might well produce a more accurate classifier

than our current approach. Finally, it might be possible to use the explanation mechanism itself to guide the design of better classifiers.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[2] C. Farabet, C. Couprie, L. Najman, and Y. LeCun, "Learning hierarchical features for scene labeling," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1915–1929, Aug. 2013.

[3] J. J. Tompson, A. Jain, Y. LeCun, and C. Bregler, "Joint training of a convolutional network and a graphical model for human pose estimation," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 1799–1807.

[4] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1–9.

[5] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural language processing (almost) from scratch," *J. Mach. Learn. Res.*, vol. 12, pp. 2493–2537, 2011.

[6] A. Bordes, S. Chopra, and J. Weston, "Question answering with subgraph embeddings," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 615–620.

[7] S. Jean, K. Cho, R. Memisevic, and Y. Bengio, "On using very large target vocabulary for neural machine translation," in *Proc. 53rd Annu. Meeting Assoc. Comput. Linguistics 7th Int. Joint Conf. Natural Lang. Process.*, 2015, pp. 1–10.

[8] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 3104–3112.

[9] T. Mikolov, A. Deoras, D. Povey, L. Burget, and J. Černock'y, "Strategies for training large scale neural network language models," in *Proc. IEEE Workshop Autom. Speech Recognit. Understanding*, 2011, pp. 196–201.

[10] G. Hinton *et al.*, "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 82–97, Nov. 2012.

[11] T. N. Sainath, A.-R. Mohamed, B. Kingsbury, and B. Ramabhadran, "Deep convolutional neural networks for LVCSR," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process.*, 2013, pp. 8614–8618.

[12] C.-S. Lee *et al.*, "Human vs. computer Go: Review and prospect," *IEEE Comput. Intell. Mag.*, vol. 11, no. 3, pp. 67–72, Aug. 2016.

[13] K. Kam, "Nvidia has doubled since March, but Ray Meyers is not selling," Forbes, 2017.

[14] S. Haykin, *Neural Networks and Learning Machines*. Upper Saddle River, NJ, USA: Prentice-Hall, 2008.

[15] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.

[16] R. Caruana, Y. Lou, J. Gehrke, P. Koch, M. Sturm, and N. Elhadad, "Intelligible models for healthcare: Predicting pneumonia risk and hospital 30-day readmission," in *Proc. 21st ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2015, pp. 1721–1730.

[17] D. Baehrens, T. Schroeter, S. Harmeling, M. Kawanabe, K. Hansen, and K.-R. Müller, "How to explain individual classification decisions," *J. Mach. Learn. Res.*, vol. 11, pp. 1803–1831, Jun. 2010.

[18] A. B. Tickle, R. Andrews, M. Golea, and J. Diederich, "The truth will come to light: Directions and challenges in extracting the knowledge embedded within trained artificial neural networks," *IEEE Trans. Neural Netw.*, vol. 9, no. 6, pp. 1057–1068, Nov. 1998.

[19] G. Bologna and Y. Hayashi, "A rule extraction study on a neural network trained by deep learning," in *Proc. Int. Joint Conf. Neural Netw.*, 2016, pp. 668–675.

[20] J. R. Zilke, E. Loza Mencía, and F. Janssen, "DeepRED—Rule extraction from deep neural networks," in *Proc. 19th Int. Conf. Discovery Sci.*, 2016, pp. 457–473.

[21] J. Yosinski, J. Clune, A. Nguyen, T. Fuchs, and H. Lipson, "Understanding neural networks through deep visualization," *ICML Workshop Deep Learn.*, 2015.

[22] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 818–833.

[23] M. D. Zeiler, D. Krishnan, G. W. Taylor, and R. Fergus, "Deconvolutional networks," in *Proc. IEEE Conf. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2010, pp. 2528–2535.

[24] L. M. Zintgraf, T. S. Cohen, T. Adel, and M. Welling, "Visualizing deep neural network decisions: Prediction difference analysis," in *Proc. Int. Conf. Learn. Represent.*, 2017.

[25] S. K. Pal and S. Mitra, "Multilayer perceptron, fuzzy sets, and classification," *IEEE Trans. Neural Netw.*, vol. 3, no. 5, pp. 683–697, Sep. 1992.

[26] J. S. R. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 3, pp. 665–685, May/Jun. 1993.

[27] J.-S. R. Jang, C.-T. Sun, and E. Mizutani, *Neuro-Fuzzy and Soft Computing: A Computational Approach to Learning and Machine Intelligence*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1997.

[28] M. Yeganejou and S. Dick, "Classification via deep fuzzy c-means clustering," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2018, pp. 1–6.

[29] Y. A. LeCun, Y. Bengio, and G. E. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[30] J. J. Rocchio, "Relevance feedback in information retrieval," in *The SMART Retrieval System—Experiments in Automatic Document Processing*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971.

[31] J. M. Keller and M. R. Gray, "A fuzzy k-nearest neighbor algorithm," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-15, no. 4, pp. 580–585, Jul./Aug. 1985.

[32] J. C. Bezdek, "Fuzzy mathematics in pattern classification," Ph.D. dissertation, Appl. Math. Center, Cornell Univ., Ithaca, NY, USA, 1973.

[33] Dataset, MNIST. [Online]. Available: http://yann.lecun.com/exdb/mnist/, Accessed: Oct. 27, 2019.

[34] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.

[35] A. Krizhevsky, "Learning multiple layers of features from tiny images," Sci. Dept., Univ. Toronto, Toronto, ON, USA, Tech. Rep. TR-2009, 2009.

[36] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. 33rd Int. Conf. Int. Conf. Mach. Learn.*, 2016, pp. 478–487.

[37] F. Tian, B. Gao, Q. Cui, E. Chen, and T.-Y. Liu, "Learning deep representations for graph clustering," in *Proc. 28th AAAI Conf. Artif. Intell.*, 2014, pp. 1293–1299.

[38] L. van der Maaten, "Learning a parametric embedding by preserving local structure," in *Proc. 12th Int. Conf. Artif. Intell. Statist.*, 2009, pp. 384–391.

[39] E. De la Rosa and Y. Wen, "Data-driven fuzzy modeling using restricted Boltzmann machines and probability theory," *IEEE Trans. Syst., Man, Cybern.: Syst.*, to be published.

[40] A. I. Aviles, S. M. Alsaleh, E. Montseny, P. Sobrevilla, and A. Casals, "A deep-neuro-fuzzy approach for estimating the interaction forces in robotic surgery," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2016, pp. 1113–1119.

[41] C. L. P. Chen, C. Y. Zhang, L. Chen, and M. Gan, "Fuzzy restricted Boltzmann machine for the enhancement of deep learning," *IEEE Trans. Fuzzy Syst.*, vol. 23, no. 6, pp. 2163–2173, Dec. 2015.

[42] Y.-J. Zheng, W.-G. Sheng, X.-M. Sun, and S.-Y. Chen, "Airline passenger profiling based on fuzzy deep machine learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 12, pp. 2911–2923, Dec. 2017.

[43] A. K. Shukla, T. Seth, and P. K. Muhuri, "Interval type-2 fuzzy sets for enhanced learning in deep belief networks," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2017, pp. 1–6.

[44] Y. J. Zheng, S. Y. Chen, Y. Xue, and J. Y. Xue, "A pythagorean-type fuzzy deep denoising autoencoder for industrial accident early warning," *IEEE Trans. Fuzzy Syst.*, vol. 25, no. 6, pp. 1561–1575, Dec. 2017.

[45] S. Rajurkar and N. K. Verma, "Developing deep fuzzy network with Takagi Sugeno fuzzy inference system," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2017, pp. 1–6.

[46] W. R. Tan, C. S. Chan, H. E. Aguirre, and K. Tanaka, "Fuzzy qualitative deep compression network," *Neurocomputing*, vol. 251, pp. 1–15, 2017.

[47] T. Zhou, F. L. Chung, and S. Wang, "Deep TSK fuzzy classifier with stacked generalization and triplely concise interpretability guarantee for large data," *IEEE Trans. Fuzzy Syst.*, vol. 25, no. 5, pp. 1207–1221, Oct. 2017.

[48] V. John, S. Mita, Z. Liu, and B. Qi, "Pedestrian detection in thermal images using adaptive fuzzy C-means clustering and convolutional neural networks," in *Proc. 14th IAPR Int. Conf. Mach. Vis. Appl.*, 2015, pp. 246–249.

[49] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*. New York, NY, USA: Wiley-Blackwell, 2000.

[50] G. J. Klir and B. Yuan, *Fuzzy Sets and Fuzzy Logic: Theory and Applications*. Upper Saddle River, NJ, USA: Prentice-Hall, Inc., 1995.

[51] J. C. Bezdek and R. J. Hathaway, "Some notes on alternating optimization," in *Proc. AFSS Int. Conf. Fuzzy Syst.*, 2002, pp. 288–300.

[52] D. Gustafson and W. Kessel, "Fuzzy clustering with a fuzzy covariance matrix," in *Proc. IEEE Conf. Decis. Control Including 17th Symp. Adapt. Processes*, 1978, pp. 761–766.

[53] F. Hoppner, F. Klawonn, R. Kruse, and T. Runkler, *Fuzzy Cluster Analysis: Methods for Classification, Data Analysis and Image Recognition*. New York, NY, USA: Wiley, 1999.

[54] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[55] T.-C. Chen, T. Stepan, S. Dick, and J. Miller, "An investigation of implicit features in compression-based learning for comparing webpages," in *Pattern Analysis and Applications*. New York, NY, USA: Springer, 2016.

[56] I. Arel, D. C. Rose, and T. P. Karnowski, "Deep machine learning—A new frontier in artificial intelligence research [Research Frontier]," *IEEE Comput. Intell. Mag.*, vol. 5, no. 4, pp. 13–18, Nov. 2010.

[57] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Philosoph. Mag. Ser.*, vol. 6, pp. 559–572, 1901.

[58] H. Stark and J. W. Woods, *Probability, Random Processes, and Estimation Theory for Engineers*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1986.

[59] L. McInnes and J. Healy, "UMAP: Uniform manifold approximation and projection for dimension reduction," 2018, *arXiv:1802.03426*.

[60] D. Silver *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.

[61] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models," 2017, *arXiv:1708.08296*.

[62] S. Hajian, F. Bonchi, and C. Castillo, "Algorithmic bias: From discrimination discovery to fairness-aware data mining," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 2125–2126.

[63] U. Fayyad, "Knowledge discovery and data mining: Towards a unifying framework," in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining*, Portland, OR, USA, 1996, pp. 82–88.

[64] B. Goodman and S. Flaxman, "European union regulations on algorithmic decision-making and a 'right to explanation'," *AI Mag.*, vol. 38, no. 3, pp. 50–57, 2017.

[65] S. Bach, A. Binder, G. Montavon, F. Klauschen, K. R. Müller, and W. Samek, "On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation," *PLoS One*, vol. 10, 2015, Art. no. e0130140.

[66] G. Montavon, S. Lapuschkin, A. Binder, W. Samek, and K. R. Müller, "Explaining nonlinear classification decisions with deep Taylor decomposition," *Pattern Recognit.*, vol. 65, pp. 211–222, 2017.

[67] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," in *Proc. Int. Conf. Learn. Represent.*, 2013.

[68] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, "Striving for simplicity: The all convolutional net," in *Proc. Int. Conf. Learn. Represent.*, 2015.

[69] A. Struyf, M. Hubert, and P. Rousseeuw, "Clustering in an object-oriented environment," *J. Statist. Softw.*, vol. 1, pp. 1–30, 1997.

[70] L. Xie, J. Wang, Z. Wei, M. Wang, and Q. Tian, "DisturbLabel: Regularizing CNN on the loss layer," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4753–4762.

[71] L. Wan, M. Zeiler, S. Zhang, Y. LeCun, and R. Fergus, "Regularization of neural networks using dropconnect," in *Proc. 30th Int. Conf. Mach. Learn.*, 2013, pp. 1058–1066.

[72] X. Chen, Y. Duan, R. Houthooft, J. Schulman, I. Sutskever, and P. Abbeel, "InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets," *NIPS*, pp. 2172–2180, 2016.

[73] P. Diehl and M. Cook, "Unsupervised learning of digit recognition using spike-timing-dependent plasticity," *Front. Comput. Neurosci.*, vol. 9, 2015, Art. no. 99.

[74] S. Zeng, B. Zhang, Y. Zhang, and J. Gou, "Collaboratively weighting deep and classic representation via l2 regularization for image classification," in *Proc. 10th Asian Conf. Machine Learn.*, 2018.

[75] Y. Zhou, K. Gu, and T. Huang, "Unsupervised representation adversarial learning network: From reconstruction to generation," in *Proc. Int. Joint Conf. Neural Netw.*, Budapest, Hungary, 2019, pp. 1–8.

[76] S. Sabour, N. Frosst, and G. Hinton, "Dynamic routing between capsules," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 3856–3866.

[77] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation policies from data," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018.

[78] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[79] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 630–645.

[80] Keras Examples, "Training ResNet on CIFAR-10 using keras," [Online]. Available: https://github.com/keras-team/keras/tree/master/keras/datasets. Accessed: Oct. 27, 2019.

[81] O. Vinyals, Y. Jia, L. Deng, and T. Darrell, "Learning with recursive perceptual representations," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 2825–2833.

[82] A. Gulati, "Understanding neurogenesis in the adult human brain," *Indian J. Pharmacol.*, vol. 47, no. 6, pp. 583–584, 2015.

[83] J. Mairal, P. Koniusz, Z. Harchaoui, and C. Schmid, "Convolutional kernel networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 2627–2635.

[84] M. Alber *et al.*, "iNNvestigate neural networks!" *J. Machine Learn. Res.*, vol. 20, no. 93, pp. 1–8, 2019.

[85] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Conf. Oper. Syst. Des. Implement.*, 2016, pp. 265–283.

[86] Y. Jia *et al.*, "Caffe: Convolutional architecture for fast feature embedding," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 675–678.

[87] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," 2012, *arXiv:1207.0580*.

[88] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[89] I. Gath and A. B. Geva, "Unsupervised optimal fuzzy clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 7, pp. 773–780, Jul. 1989.

[90] V. Losing, B. Hammer, and H. Wersing, "Incremental on-line learning," *Neurocomput.*, vol. 275, no. C, pp. 1261–1274, Jan. 2018.

[91] Market Capitalization of NVIDIA. [Online]. Available: https://www.forbes.com/companies/nvidia/. Accessed: Oct. 27, 2019.

[92] A. I. Google, Google Brain Team. [Online]. Available: https://ai.google/research/teams/brain/. Accessed: Oct. 27, 2019.